Аутентификация во frontend-приложении

Защищённые страницы, навигация по ролям и переходы в Internet Shop

Зачем нужна эта лекция

- ЛР-5 уже реализовала login(), apiFetch(), authState.js и logout.
- Теперь интерфейс должен показывать разные страницы гостю, пользователю и администратору.
- Разберём навигацию и проверки на frontend без повторного написания JWT-логики.

Что научимся делать

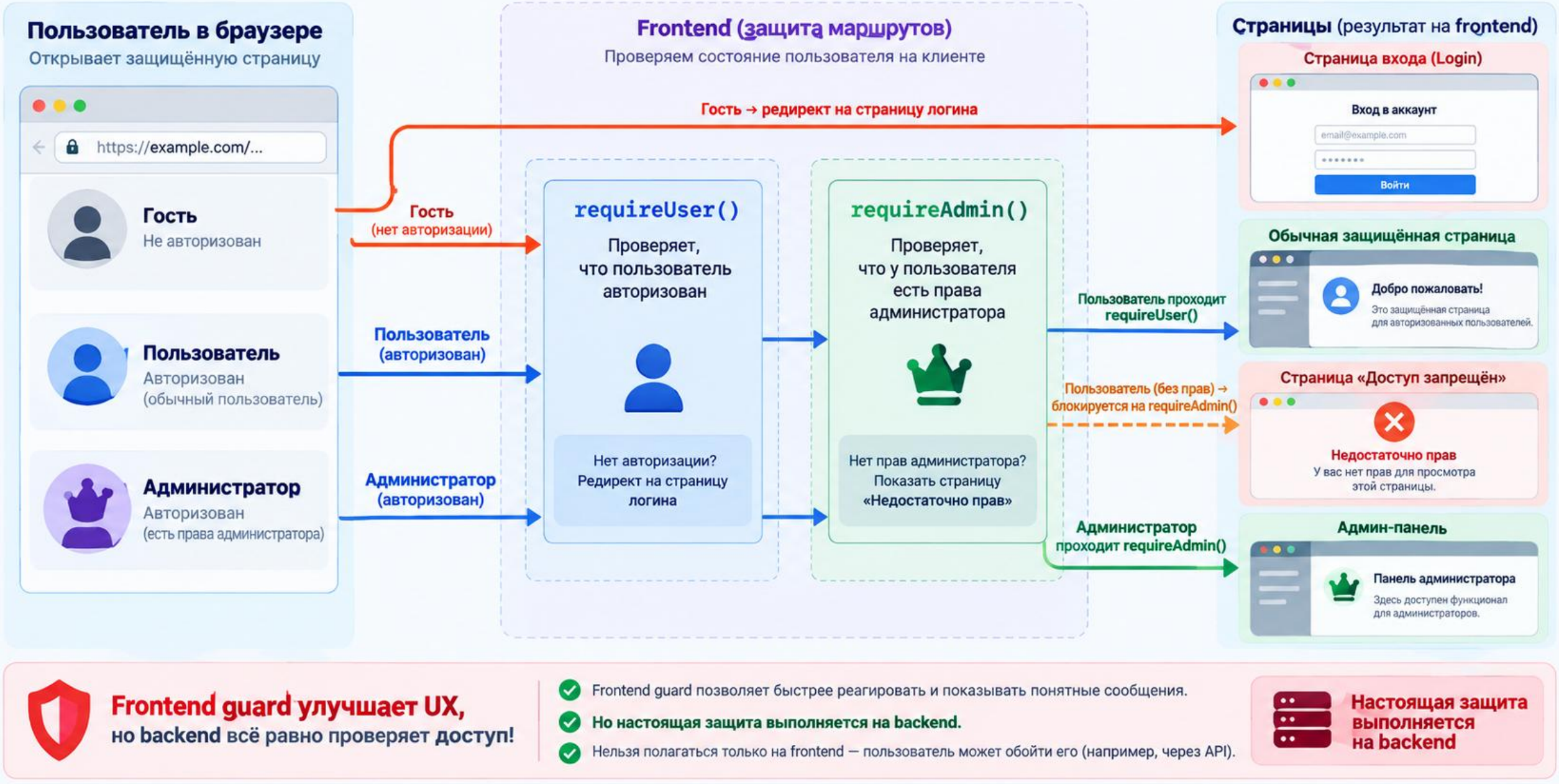
- Разделять публичные и защищённые разделы интерфейса.
- Проверять состояние пользователя перед показом страницы.
- Учитывать роль user/admin при отображении меню.
- Подключать простую навигацию к существующему frontend.

Какие страницы доступны

Раздел	Гость	user	admin
Каталог	Да	Да	Да
Вход	Да	Не требуется	Не требуется
Мой профиль	Нет	Да	Да
Панель управления	Нет	Нет	Да

Protected page guard: guest → login, user → page, admin → admin page

Как работает защита страниц на frontend (например, в React / Vue / Angular)



Frontend guard улучшает UX, но backend всё равно проверяет доступ!

- Frontend guard позволяет быстрее реагировать и показывать понятные сообщения.
- **Но настоящая защита выполняется на backend.**
- Нельзя полагаться только на frontend — пользователь может обойти его (например, через API).

Настоящая защита выполняется на backend

Что такое protected page

- Protected page — раздел интерфейса, который показывается только пользователю после входа.
- Например: профиль и раздел заказов.
- Ограничение интерфейса не заменяет защиту REST API на сервере.

Что такое route guard

- Route guard — проверка перед показом раздела.
- Если пользователь не вошёл — показываем вход.
- Если роль не подходит — показываем «Недостаточно прав».
- Это функция управления интерфейсом, а не средство защиты данных.

Используем существующий authState.js

- В ЛР-5 уже созданы getCurrentUser(), getAccessToken(), setAuth() и clearAuth().
- Не переписываем эти функции: проверяем результат getCurrentUser().
- Если функция возвращает null — пользователь не вошёл.

Проверяем вход перед открытием страницы

```javascript
import { getCurrentUser } from "./authState.js";

function requireUser() {
  const user = getCurrentUser();
  if (!user) {
    location.hash = "#/login";
    return false;
  }
  return true;
}
```

Что важно

- getCurrentUser() берёт сведения из уже готового модуля.
- false означает, что показывать защищённый раздел нельзя.
- location.hash меняет часть URL после #.

Проверяем роль администратора

```javascript
function requireAdmin() {
  const user = getCurrentUser();
  if (!user) {
    location.hash = "#/login";
    return false;
  }
  if (user.role !== "admin") {
    alert("Недостаточно прав");
    return false;
  }
  return true;
}
```

Что важно

- Сначала проверяем факт входа, затем роль.
- user.role берётся из текущего состояния интерфейса.
- Backend по-прежнему обязан проверять requireAuth и requireRole.

Навигация через hash — простейший вариант

- URL вида /#/catalog, /#/login, /#/profile, /#/admin.
- По изменению hash выполняем функцию renderPage().
- Переходы выполняются без повторной загрузки HTML-документа.
- Для учебной работы достаточно одной HTML-страницы.

Определяем текущий раздел

```javascript
function renderPage() {
  const page = location.hash || "#/catalog";
  if (page === "#/profile" && !requireUser()) return;
  if (page === "#/admin" && !requireAdmin()) return;
  showPage(page);
}

window.addEventListener("hashchange", renderPage);
renderPage();
```

Что важно

- showPage() — функция, которую реализуем для переключения HTML-секций.
- hashchange срабатывает при смене части URL после #.
- При открытии страницы запускаем renderPage() один раз.

Показываем только нужный раздел

```javascript
function showPage(page) {
  document.querySelectorAll("[data-page]").forEach(se
    section.hidden = section.dataset.page !== page;
  });
}
```

Что важно

- Каждой секции назначаем data-page, например #/catalog.
- Свойство hidden скрывает остальные разделы.
- Этот код не запрещает прямое обращение к backend API.

HTML-разделы внутри одной страницы

```HTML
<section data-page="#/catalog">Каталог</section>
<section data-page="#/login" hidden>Вход</section>
<section data-page="#/profile" hidden>Профиль</sectio
<section data-page="#/admin" hidden>Панель admin</se
```

Что важно

- Это фрагмент HTML, не JavaScript.
- Разделы используются функцией showPage().
- В существующем frontend можно применять отдельные section или адаптировать текущие.

Навигация зависит от роли

```javascript
function renderNavigation() {
  const user = getCurrentUser();
  loginLink.hidden = Boolean(user);
  profileLink.hidden = !user;
  adminLink.hidden = user?.role !== "admin";
  logoutButton.hidden = !user;
}
```

Что важно

- Не вошёл: видна ссылка «Войти».
- Вошёл: доступны профиль и выход.
- Только admin видит ссылку панели управления.

Что происходит после login

- Сервер возвращает user и accessToken — как в ЛР-5.
- setAuth(data) уже сохраняет эти значения.
- Теперь после входа вызываем renderNavigation() и renderPage().
- Можно перейти к #/profile или другому защищённому разделу.

Обновление интерфейса после успешного входа

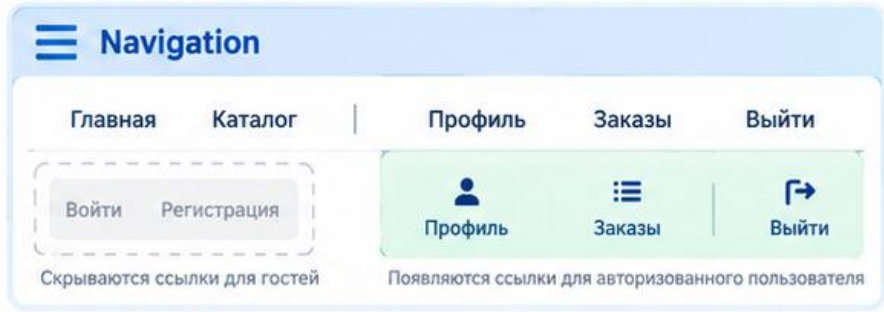
```javascript
const user = await login(emailInput.value, passwordIn
renderNavigation();
location.hash = "#/profile";
```

Что важно

- login() уже реализована в ЛР-5.
- Повторно писать запрос через fetch() не требуется.
- Навигация обновляется после успешной авторизации.

Frontend auth lifecycle: login → state → protected request → response

От входа пользователя до обновления интерфейса



Интерфейс меняется после login

Login

Email

user@example.com

Password

••••••••

Войти

Пользователь вводит credentials

POST /api/auth/login

```
fetch('/api/auth/login', {
  method: 'POST',
  headers: {
    'Content-Type':
     'application/json'
  },
  body: JSON.stringify({
    email, password
  })
})
```

Frontend отправляет login request

Ответ backend

```
{
  "user": {
    "id": 1,
    "name": "Иван",
    "email": "user@example.com"
  },
  "accessToken":
  "eyJhbGciOiJIUzI1NiIsInR5cCI6…"
}
```

Backend возвращает пользователя и access token

authState

```
{
  status: 'authenticated',
  user: {
    id: 1,
    name: 'Иван',
    email: 'user@example.com"
  },
  accessToken:
  'eyJhbGciOiJIUzI1NiIsInR5cCI6…"
}
```

Frontend сохраняет auth state

Protected request

GET /api/orders/my

```
apiFetch('/api/orders/my',{
  method: 'GET',
  headers: {
    'Authorization':
    'Bearer <accessToken>'
  }
})
```

apiFetch() добавляет Bearer token

Protected API

Express JS

GET /api/orders/my (protected)

- Проверяет JWT
- Извлекает данные пользователя
- Проверяет права доступа

Backend проверяет JWT

JSON Response

```
[
  {
    "id": 101,
    "title": "Ноутбук",
    "status": "Доставлен",
    "total": 59900
  },
  {
    "id": 102,
    "title": "Клавиатура",
    "status": "В пути",
    "total": 8900
  }
]
```

Данные возвращаются с сервера

UI

Мои заказы

- Ноутбук 59 900 ₽ — Доставлен
- Клавиатура 8 900 ₽ — В пути
- Наушники 4 500 ₽ — Обработка

Frontend обновляет DOM / UI

Ключевая идея:

1. Login создаёт authenticated state
2. **authState** управляет navigation и protected requests
3. **Backend** остаётся источником истины

Что делать при выходе и ошибках

- При logout используем существующий clearAuth() и обновляем интерфейс.
- При 401 в ЛР-5 уже очищается состояние пользователя.
- При 403 показываем сообщение, но не выполняем logout.
- Нужно скрыть персональные данные на ранее открытых страницах.

Почему после F5 снова появляется вход

- JWT в учебном проекте хранится только в памяти JavaScript.
- После перезагрузки страницы переменные сбрасываются.
- Поэтому пользователь становится гостем.
- Для этой лабораторной восстановление сессии не реализуем.

Что такое auth bootstrap — только для знакомства

- Auth bootstrap — проверка действующей серверной сессии при открытии приложения.
- Чаще используется с серверной сессией или HttpOnly cookie.
- В текущей учебной реализации с JWT только в памяти bootstrap не требуется.

Почему frontend-проверок недостаточно

- Пользователь может изменять JavaScript и вручную отправлять HTTP-запросы.
- Скрытая кнопка и route guard защищают только интерфейс.
- Backend должен проверять JWT, роль и владение данными независимо от frontend.

Типичные результаты проверки

Сценарий	Интерфейс	Проверка сервера
Гость открывает профиль	Переход на вход	JWT необходим
user открывает admin	Сообщение о запрете	403 на admin API
admin открывает admin	Панель показана	JWT + роль admin
JWT истёк	Переход на вход	401

Мини-практика для Internet Shop

- Взять существующий static/ из ЛР-5; новый frontend не создавать.
- Добавить ссылки «Каталог», «Профиль», «Панель admin».
- Реализовать showPage(), requireUser(), requireAdmin(), renderNavigation().
- Проверить гостя, обычного пользователя и администратора.

Полный frontend authentication lifecycle — Internet Shop

От запуска приложения до выхода из системы. Основные состояния, запросы и ответы (200 / 401 / 403).

Internet Shop
Learn. Build. Deploy.

1 Application start
Запуск приложения

Internet Shop

Загрузка приложения...

Инициализация frontend
Создаём пустой authState (loading) и проверяем аутентификацию (например, /api/auth/me для cookie-based).

2 Auth state
Определение состояния

```
authState = {
  status: 'loading',
  user: null,
  accessToken: null
}
```

Гость (guest)
Пользователь не аутентифицирован

Аутентифицирован (authenticated)
Есть данные пользователя

На основе ответа /api/auth/me или наличия токена в памяти

3 Login
Вход в систему

Вход в аккаунт

user@example.com

••••••••

Войти

POST /api/auth/login

```
{
  "email": "user@example.com",
  "password": "••••••••"
}
```

4 Authenticated
Сохраняем данные

Ответ сервера (200 OK)

```
{
  "user": {
    "id": 1,
    "name": "Иван",
    "role": "user"
  },
  "accessToken": "eyJhbGci0i..."
}
```

Обновляем authState (в памяти)

```
authState = {
  status: 'authenticated',
  user: { id: 1, name: 'Иван',
    role: 'user'},
  accessToken: 'eyJhbGci0i...'
}
```

5 Protected pages
Доступ к защищённым страницам

Internet Shop

Главная Каталог Мои заказы Профиль

Мои заказы

Заказ #1001 12.09.2026	Доставлен
Заказ #1002 10.09.2026	В обработке
Заказ #1003 05.09.2026	Отменён

Перед загрузкой страницы (frontend guard) проверяем authState. Гость → перенаправляем на /login.
Пользователь → открываем страницу.
Роль (admin) → дополнительные проверки.

7 apiFetch()
Запрос к API

```
GET /api/orders/my
Headers:
 Authorization:
  Bearer <accessToken>
```

Express.js
+ Prisma
+ PostgreSQL

1. Проверка токена (JWT)
2. Определение пользователя
3. Проверка прав (role)
4. Обработка запроса
5. Возврат JSON

8 Ответ API
Возможные варианты

200 OK

```
{
  "orders": [ ... ]
}
```

Данные успешно получены

401 Unauthorized

```
{
  "error": "Unauthorized"
}
```

Токен отсутствует, недействителен или истёк

403 Forbidden

```
{
  "error": "Forbidden"
}
```

Пользователь аутентифицирован, но нет прав

9 UI / Отображение
Показываем данные или ошибки

Список заказов

Сообщение об ошибке

Сессия истекла. Войдите в систему.

Перейти на страницу входа

Недостаточно прав

У вас нет прав для выполнения этого действия.

10 Logout / Reload
Выход из системы или перезагрузка страницы

Выход (logout)
Очищаем authState (и, при необходимости, уведомляем backend)

```
authState = { status: 'guest', user:null, accessToken: null }
```

Перезагрузка страницы (F5)
Вся JavaScript-память очищается. Приложение снова стартует с loading, и процесс повторяется.

Основные состояния authState

loading → Проверяем аутентификацию

guest → Пользователь не вошёл в систему

Login (200) / Logout / Reload

authenticated → Есть пользователь и accessToken

Коды ответов и их смысл

200 OK	Запрос выполнен, возвращаются данные
401 Unauthorized	Токен отсутствует, недействителен или истёк
403 Forbidden	Пользователь аутентифицирован, но нет прав

Важно помнить

- Route guard во frontend улучшает UX, но не заменяет проверку на backend.
- Токен хранится только в памяти (или в cookie при BFF).
- После перезагрузки страницы состояние теряется.
- Все защищённые запросы всегда проверяются на backend.
- Используйте правильные коды ответов: 200 / 401 / 403.

Результат

Безопасный и понятный жизненный цикл аутентификации во frontend-приложении Internet Shop.

Итоги занятия

1. Frontend управляет отображением разделов по auth state и role.
2. Route guard решает, какой экран показать, но не защищает REST API.
3. В курсовой роли и ограничения доступа обязательно проверяются на backend.
4. Используем существующие apiFetch(), login(), authState.js и logout.
5. При F5 JWT из памяти исчезает — в учебной версии это ожидаемо.

Контрольные вопросы

1. Что такое protected page и route guard?
2. Почему скрытие ссылки admin не защищает API?
3. Чем requireAdmin() во frontend отличается от requireRole("admin") на backend?
4. Что делает location.hash?
5. Почему после F5 пользователь снова видит вход?